

Data Structures for Placements — Day 2: Strings & Sorting/Searching

DSA Prep Notes

oday

Day 2: Strings + Sorting & Searching Basics

1. Strings in C++ — Theory

A string is a sequence of characters. C++ has two flavors:

- **C-style string:** `char arr[]` terminated by `'\0'`
- **`std::string`:** a dynamic, resizable class (internally similar to `vector<char>`)

Key properties: `std::string` is **mutable** in C++ (unlike Java/Python, where strings are immutable) — you can modify characters in place via `str[i] = 'x'`. Being contiguous in memory, indexing is $O(1)$, just like arrays.

Complexity of Common String Operations

Operation	Complexity	Why
Access by index	$O(1)$	Contiguous memory
Traversal	$O(n)$	
Comparison	$O(n)$	Char-by-char worst case
Concatenation	$O(n + m)$	Copies both into new buffer
Substring	$O(k)$, k = substring length	Copies characters
Search substring (naive)	$O(n*m)$	
Search substring (KMP/Z-algo)	$O(n + m)$	
Reverse	$O(n)$	

Gotcha: repeated concatenation in a loop can degrade to $O(n^2)$ in the worst case due to reallocations — same amortized-growth reasoning as vectors.

2. Sorting Algorithms — Theory & Comparison

Stability = do equal elements retain their relative input order after sorting? This matters when sorting objects by one key while needing a secondary order preserved.

Algorithm	Best	Average	Worst	Space	Stable?	In-place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes

How Each Works

- **Bubble Sort:** repeatedly swap adjacent out-of-order elements. An early-exit flag (**swapped**) gives it $O(n)$ best case on already-sorted input.
- **Selection Sort:** repeatedly find the minimum of the remaining unsorted portion and swap it into place. Always $O(n^2)$, regardless of input order — no early exit possible.
- **Insertion Sort:** build a sorted prefix one element at a time, shifting larger elements right to make room. Efficient ($O(n)$) on nearly-sorted input — this is why it's often used as the base case for hybrid sorts (e.g. Timsort) on small subarrays.
- **Merge Sort:** divide and conquer — split the array in half recursively, sort each half, then merge two sorted halves using an auxiliary array. Guarantees $O(n \log n)$ in all cases, but needs $O(n)$ extra space for merging.
- **Quick Sort:** pick a pivot, partition the array so smaller elements go left and larger go right of the pivot, then recurse on both sides. In-place (no auxiliary array needed for partitioning), but degrades to $O(n^2)$ when the pivot choice is consistently bad (e.g. always picking the last element on an already-sorted array).

Why Quick Sort's worst case is $O(n^2)$: if the pivot is always the smallest/largest remaining element, each partition step removes only one element, leading to n recursive calls each doing $O(n)$ work. **Interview tip:** mention randomized pivot selection to avoid this on adversarial/sorted inputs.

Why Merge Sort needs $O(n)$ extra space: the merge step needs a temporary array to hold merged results before copying back, unlike Quick Sort's in-place partitioning.

Full tested implementations of all 5 algorithms are in `01_sorting_algorithms.cpp`.

Solved Problems — Deep Dive

Problem 1: Check if a String is a Palindrome

Optimal — Two Pointer, $O(n)$ time, $O(1)$ space: compare characters from both ends moving inward; mismatch means not a palindrome.

Edge cases: empty string (trivially true), single character (true), mixed case (normalize with `tolower()` if case-insensitive comparison is required), spaces/punctuation (real-world variants like “A man a plan a canal Panama” require stripping non-alphanumeric characters first).

Problem 2: Check if Two Strings are Anagrams

Brute force — Sort & Compare, $O(n \log n)$: sort both strings and check equality.

Optimal — Frequency Count, $O(n)$ time, $O(1)$ space (fixed 26-size array for lowercase letters): increment counts for the first string, decrement for the second; if all counts return to zero, they’re anagrams.

Dry run: a=“listen”, b=“silent” $\rightarrow$ all frequency counts cancel to 0 $\rightarrow$ true.

Edge cases: different lengths (early return false), case sensitivity (normalize if needed), non-alphabetic or mixed-case input (use `unordered_map<char,int>` instead of a fixed 26-array for general Unicode/mixed-case support).

Problem 3: First Non-Repeating Character in a String

Brute force: for each character, scan the whole string to count occurrences — $O(n^2)$.

Optimal — Frequency Map, $O(n)$ time, $O(1)$ space (bounded alphabet): build a frequency array in one pass, then scan again to find the first character with frequency exactly 1.

Dry run: s = “swiss” $\rightarrow$ frequencies: s=3, w=1, i=1 $\rightarrow$ second pass finds ‘s’ has freq 3 (skip), ‘w’ has freq 1 $\rightarrow$ return ‘w’.

Edge cases: all characters repeat (return a sentinel value), empty string, single character (it is the answer by definition).

Problem 4: Search in a Rotated Sorted Array

A classic FAANG-interview favorite. Given a sorted array that’s been rotated at an unknown pivot (e.g. [4,5,6,7,0,1,2]), find a target in $O(\log n)$.

Key insight: at any point during binary search, at least one half (left of mid, or right of mid) is guaranteed to be normally sorted. Determine which half is sorted, check whether the target falls within that half’s range, and recurse into the correct side accordingly.

Dry run: arr=[4,5,6,7,0,1,2], target=0. - low=0, high=6, mid=3 (val=7). Left half [4,5,6,7] is sorted. Is $4 \leq 0 < 7$? No $\rightarrow$ search right half, low=4. - low=4, high=6, mid=5 (val=1). Left half [0,1] is sorted. Is $0 \leq 0 < 1$? Yes $\rightarrow$ search left half, high=4. - low=4, high=4, mid=4 (val=0). Match found — return index 4.

Edge cases: target not present (return -1), array not actually rotated (algorithm still works correctly — one side is always “fully sorted”), duplicate elements (this classic version assumes

distinct elements; with duplicates, `arr[low] <= arr[mid]` can't reliably determine the sorted half, and the worst case degrades to $O(n)$ — a common follow-up question interviewers ask).

Summary Table — Day 2

Problem	Optimal Approach	Time	Space
Palindrome Check	Two Pointer	$O(n)$	$O(1)$
Anagram Check	Frequency Count	$O(n)$	$O(1)$
First Non-Repeating Char	Frequency Map	$O(n)$	$O(1)$
Search Rotated Sorted Array	Modified Binary Search	$O(\log n)$	$O(1)$
Sorting (general purpose, guaranteed)	Merge Sort	$O(n \log n)$	$O(n)$
Sorting (general purpose, in-place, avg case)	Quick Sort	$O(n \log n)$ avg	$O(\log n)$

Code files for today: `01_sorting_algorithms.cpp`, `02_strings_searching_problems.cpp` — both compiled and tested.

Next session (Day 3): Linked Lists — Singly, Doubly, Circular.